

ECL v0.1 Technical Report

Title: ECL: A Deterministic Execution IR for Replayable Agent Runtime Traces

Author: Bin Zhang

Affiliation: Independent researcher

Version: 0.1

Date: 2026-06-30

Software archive: <https://zenodo.org/records/21003766>

Version DOI: [10.5281/zenodo.21003766](https://doi.org/10.5281/zenodo.21003766)

Concept DOI: [10.5281/zenodo.21003765](https://doi.org/10.5281/zenodo.21003765)

Repository: <https://github.com/joy7758/ecl-execution-compact-layer>

Abstract

Agent runtimes expose execution traces through framework-specific surfaces such as model inputs, tool calls, callbacks, spans, run trees, and event streams. These traces are useful inside their host frameworks, but they are difficult to compare, replay, or cite across runtime boundaries. Execution Compact Layer (ECL) v0.1 is a deterministic execution intermediate representation for agent runtime traces. It maps supported OpenAI-style and LangChain-style traces into four replayable surfaces: **state**, **intent**, **action**, and **evidence**. The v0.1 artifact includes a frozen schema, validator, replay scripts, loss-aware adapters, an embeddable SDK, a dependency API, and an MCP-shaped local wrapper. ECL is not a framework, public standard, production audit system, or proof of external adoption. It is a small software artifact for local validation, replay, hashing, and citation of normalized agent execution records.

1. Problem

Modern agent systems produce rich traces, but those traces are usually tied to the runtime that created them. This makes it hard to answer a narrow reproducibility question:

Can a runtime trace be represented as a small execution object that can be validated, replay

ECL addresses this question by defining a compact execution representation rather than a full observability or provenance system.

2. Execution IR

An ECL record is an execution object:

E = (state, intent, action, evidence)

The four surfaces are:

- **state**: execution lifecycle, actor, runtime identity, and correlation references.
- **intent**: requested operation, constraints, expected result, and evidence requirements.
- **action**: tool or operation step, execution mode, side-effect class, and parameter hash.
- **evidence**: result status, trace references, event chain, policy decisions, and artifact hashes.

The machine-readable schema is:

`schemas/ecl-execution-compact-layer.schema.json`

The IR is intentionally smaller than a host-runtime trace. It preserves the information needed by ECL's validation and replay contract, and records mapping loss when source information is not fully preserved.

3. Deterministic Execution Model

The ECL v0.1 execution flow is:

```
runtime trace
-> normalize/adapt
-> ECL record
-> validate
-> replay
-> {execution_trace, evidence_bundle, replay_result}
-> artifact hashes
```

For unchanged input trace, schema, adapter behavior, validator, replay function, checkpoint reference, and generation parameters, the following values are expected to remain stable:

- ECL object hash
- validation result
- execution trace hash
- evidence bundle hash
- replay result hash

This replay property verifies representational stability. It does not prove that a real-world side effect occurred outside the source runtime.

4. Cross-Runtime Mapping

ECL v0.1 implements local mapping for two trace families:

- OpenAI Agents SDK-style traces
- LangChain-style traces

The adapter rule is:

Runtime source	ECL surface
runtime metadata, actor, correlation	state
model input, reasoning, requested operation	intent
tool call or run step	action
events, outputs, result references	evidence

Mapping is loss-aware. When source information is omitted, approximated, or structurally remapped, adapters record a loss report rather than claiming full-fidelity preservation.

5. SDK and Dependency Interface

The SDK exposes a compact local API:

```
from sdk import ECL

record = ECL.create(state, intent, action)
ECL.validate(record)
ECL.replay(record)
```

The dependency interface exposes a non-invasive adoption surface:

```
import ecl_dependency as ecl

ecl_object = ecl.wrap(trace)
payload = ecl.emit(ecl_object)
result = ecl.verify(ecl_object)
```

This lets a host project wrap or verify an execution trace without modifying the host runtime.

6. MCP-Shaped Local Wrapper

ECL v0.1 includes:

```
mcp/ecl_tool_spec.json
mcp/ecl_server_stub.py
```

The local wrapper exposes:

- `ecl.wrap`
- `ecl.emit`
- `ecl.verify`

This is an MCP-shaped local surface only. It is not a published MCP server, registry plugin, JSON-RPC transport, authorization layer, or external adoption signal.

7. Local Evaluation

The local verification command is:

```
python3 -m unittest discover -s tests
```

The checked repository state records:

```
Ran 78 tests
OK
```

The synthetic trace-corpus evaluation records:

```
python3 experiments/evaluate_trace_corpus.py
case_count=12
by_runtime={"langchain": 6, "openai": 6}
valid_count=12
deterministic_count=12
loss_expectation_met_count=12
surface_coverage={"action": 12, "evidence": 12, "intent": 12, "state": 12}
loss_type_counts={"semantic": 9, "structural": 3}
evaluation_hash=sha256:2434da21056811e7eacf1ffac6944d5420ddeb2aa783f74423326a2b54a33974
```

The validation-matrix evaluation records:

```
python3 experiments/evaluate_validation_matrix.py
baseline_valid=true
case_count=8
expected_invalid_count=8
invalid_detected_count=8
expectation_met_count=8
evaluation_hash=sha256:00fe0ee8952988f7460280b40554889a890c93f9d181a6c9d8ee8b0194b031c0
```

The field-level mapping coverage evaluation records:

```
python3 experiments/evaluate_mapping_coverage.py
case_count=12
total_source_fields=81
direct_mapped_field_count=80
source_hash_only_field_count=1
loss_missing_field_count=4
evaluation_hash=sha256:8a8b820ecbdd1b4e88a2d8e07b05be2d479add0f0c6c3265292cc5a86763e43a
```

These evaluations use included local synthetic traces. They are not production trace evidence, third-party validation, benchmark superiority, or external adoption.

8. Abstraction Boundary

ECL is not reducible to a tracing backend, a wrapper, or a JSON schema plus hashes. The irreducible v0.1 core is:

```
runtime trace
-> deterministic adapter
-> ECL object: state / intent / action / evidence
-> validation
-> replay artifacts
-> stable hashes
-> explicit loss report when preservation is incomplete
```

This makes ECL narrower than broad observability or provenance systems, but more specific about deterministic local replay.

9. Related Systems

ECL is adjacent to but not a replacement for:

- OpenTelemetry: distributed telemetry traces and spans.
- LangSmith / LangChain tracing: framework observability and run tracing.
- W3C PROV: general provenance interchange.
- OpenLineage: dataset and job lineage metadata.
- OpenAI Agents tracing: OpenAI Agents SDK workflow tracing.

ECL consumes or normalizes selected trace shapes. It does not attempt to preserve all information from these systems and is not a superset of them.

10. Limitations

- ECL v0.1 supports only included OpenAI-style and LangChain-style trace fixtures.
- The evaluations are local and synthetic.
- The MCP-shaped wrapper is local only and not a conformant server.
- The Zenodo DOI records a software archive, not peer review.
- The Journal of Systems & Software submission was rejected at pre-screening and is closed.
- arXiv submission remains blocked by endorsement status and has no arXiv ID.
- No external adoption, production deployment, benchmark leadership, or standards-body endorsement is claimed.

11. Citation

Zhang, B. (2026). ECL v0.1 (0.1). Zenodo. <https://doi.org/10.5281/zenodo.21003766>

References

- ECL repository: <https://github.com/joy7758/ecl-execution-compact-layer>
- ECL Zenodo record: <https://zenodo.org/records/21003766>
- ICSE 2027 Tool Demonstration and Data Showcase: <https://conf.researchr.org/track/icse-2027/icse-2027-demonstrations>